

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验 4 ARM 中断实验]

姓名：	赵瑞杰
学号：	216001944
课程名称：	嵌入式系统
所在学院：	信息学院
专业班级：	计算机科学与技术 1 班
任课教师：	王翀 王艺睿

实验项目名称	ARM 中断实验

一、实验目的与要求：

示例将利用 Exynos4412 的 K2、K3 这 2 个 I/O 引脚的中断模式，当被按下时进入相应的中断处理函数处理相应的事件。

二、实验环境：

Windows 7 操作系统、ARM Cortex-A9 FS4412 实验箱、Eclipse for ARM

三、实验方法（原理、流程图）

1、电路原理

电路原理如图所示，K2、K3 分别与 GPX1_1、GPX1_2 相连，在没有按下按键时 GPX1_1、GPX1_2 引脚上一直处于高电平，当把这两个引脚设为中断模式并为下降沿中断，则按键被按下俩引脚就会有高电平变为低电平，因此，产生 GPIO 中断进入相应的中断函数，处理中段事件，从终端上打印出相应的按键信息。其中 K2 对应的是 XEINT9 中断源，K3 对应的是 XEINT10 中断源。

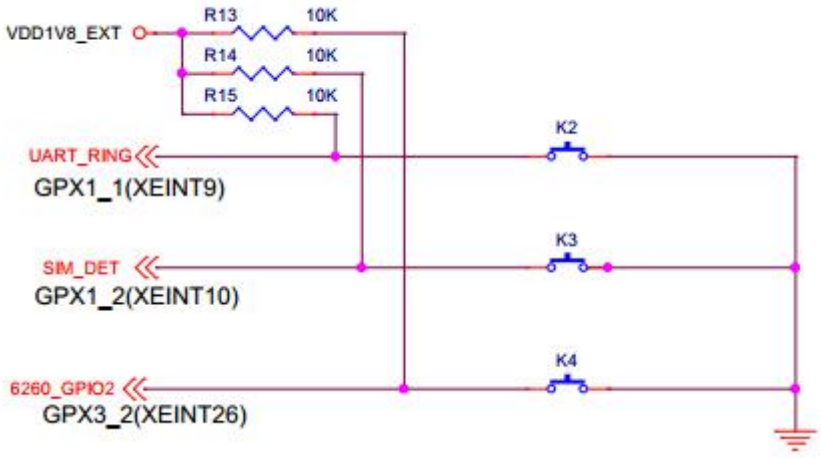


图 EXYNOS4412 中断实验电路图

2、编程流程

（1）设置 GPX1_2、GPX1_2 两个管脚没有内部上下拉属性，然后配置为中断模式；

（2）设置中断触发方式；

（3）GPIO 控制器中关闭屏蔽、使能中断；

（4）在 GIC 中断控制器中使能中断；

（5）设置中断优先级；

（6）使能 GIC；

（7）选择中断发送给 CPU0；

（8）等待中断产生，然后进入中断处理器函数；

（9）清楚中断源的挂起状态。

四、实验过程、步骤及内容

1、 寄存器设置

为了实现进入中断目的，需要通过配置 GPX1CON 寄存器将 GPX1_1、GPX1_2 设置为中断模式。设置中断方式、中断处理函数、使能中断。

2、 程序编写

相关代码如下：

（1） 设置异常向量表

注意：`C12` 寄存器用于设定向量表的基地址，这个地址和 `map.lds` 中的连接地址一致，是 `0x40008000`，可以更改测试。

```
.text
.global _start
_start:
        b       reset
        ldr     pc,_undefined_instruction
        ldr     pc,_software_interrupt
        ldr     pc,_prefetch_abort
        ldr     pc,_data_abort
        ldr     pc,_not_used
```

```
10          ldr     pc,_irq
11          ldr     pc,_fiq
12
13  _undefined_instruction: .word  _undefined_instruction
14  _software_interrupt:    .word  _software_interrupt
15  _prefetch_abort:        .word  _prefetch_abort
16  _data_abort:            .word  _data_abort
17  _not_used:              .word  _not_used
18  _irq:                   .word  irq_handler
19  _fiq:                   .word  _fiq
20
21  reset:
22
23      ldr r0,=0x40008000
24      mcr p15,0,r0,c12,c0,0       @ Vector Base Address Register
25      ……
```

（2） 编写中断处理函数

汇编中的处理

```
1       /****  irq_handler  ****/
2   irq_handler:
3
4       sub  lr,lr,#4
5       stmfd sp!,{r0-r12,lr}
6       bl  do_irq   //跳转到 do_irq 处理器函数
7       ldmfd sp!,{r0-r12,pc}^
```

主程序中的中断初始化

```
1   /*-------------------------MAIN FUNCTION-----------------------------*/
2   /*********************************************************************
3    * @brief       Main program body
4    * @param[in]   None
5    * @return      int
6    *********************************************************************/
7   int main(void)
8   {
9       //LED2 GPX2_7
10      GPX2.GPX2CON |= 0x1 << 28;
11      //LED3 GPX1_0
12      GPX1.GPX1CON |= 0x1;
13      //Led4 GPF3_4
14      GPF3.GPF3CON |= 0x1 << 16;
```

```
15
16          //Key_2  Interrupt  GPX1_1
17          GPX1.GPX1PUD = GPX1.GPX1PUD & ~(0x3 << 2); // Disables Pull-up/Pull-d
18own
19          GPX1.GPX1CON = (GPX1.GPX1CON & ~(0xF << 4)) | (0xF << 4); //GPX1_1: W
20AKEUP_INT1[1](EXT_INT41[1])
21          EXT_INT41_CON = (EXT_INT41_CON & ~(0x7 << 4)) | 0x2 << 4;
22          EXT_INT41_MASK = (EXT_INT41_MASK & ~(0x1 << 1)); //  Bit: 1 = Enables
23 interrupt
24
25          //Key_3  Interrupt  GPX1_2
26          GPX1.GPX1PUD = GPX1.GPX1PUD & ~(0x3 << 4); // Disables Pull-up/Pull-d
27own
28          GPX1.GPX1CON = (GPX1.GPX1CON & ~(0xF << 8)) | (0xF << 8); //GPX1_2:WA
29KEUP_INT1[2] (EXT_INT41[2])
30          EXT_INT41_CON = (EXT_INT41_CON & ~(0x7 << 8)) | 0x2 << 8;
31          EXT_INT41_MASK = (EXT_INT41_MASK & ~(0x1 << 2)); //  Bit: 1 = Enables
32 interrupt
33
34
35          /*
36           * GIC interrupt controller:
37           * */
38
39          // Enables the corresponding interrupt SPI25, SPI26 -- Key_2, Key_3
40          ICDISER.ICDISER1 |= (0x1 << 25) | (0x1 << 26);
41
42          CPU0.ICCICR |= 0x1; //Global enable for signaling of interrupts
43
44          CPU0.ICCPMR = 0xFF; //The priority mask level.Priority filter. thresh
45old
46
47          ICDDCR = 1;      //Bit1:  GIC monitors the peripheral interrupt signal
48s and
49                           //      forwards pending interrupts to the CPU interf
50aces2
51
52          ICDIPTR.ICDIPTR14 = 0x01010101; //SPI25  SPI26  interrupts are sent t
53o processor 0
54
55          printf("\n ********* GIC test ********\n");

          while (1){
              GPF3.GPF3DAT |= 0x1 << 4;
```

```
            mydelay_ms(500);
            GPF3.GPF3DAT &= ~(0x1 << 4);
            mydelay_ms(500);
        }

        return 0;
    }
```

中断处理函数

```
/*********************************************************************
 * @brief       IRQ Interrupt Service Routine program body
 * @param[in]   None
 * @return      None
 *********************************************************************/
void do_irq(void )
{
    int irq_num;
    irq_num = (CPU0.ICCIAR & 0x1FF);
    switch (irq_num) {

    case 58: //turn on LED2; turn off LED3
        GPX2.GPX2DAT = 0x1 << 7;
        GPX1.GPX1DAT &= ~0x1;
        printf("IRQ interrupt !! turn on LED2; turn off LED3\n");

        //Clear Pend
        EXT_INT41_PEND |= 0x1 << 2;
        ICDICPR.ICDICPR1 |= 0x1 << 26;
        break;
    case 57: //Turn on Led3; Turn off Led2

        GPX2.GPX2DAT &= ~(0x1 << 7);
        GPX1.GPX1DAT |= 0x1;
        printf("IRQ interrupt !! Turn on LED3; Turn off LED2\n");

        //Clear Pend
        EXT_INT41_PEND |= 0x1 << 1;
        ICDICPR.ICDICPR1 |= 0x1 << 26;
        break;
    }

    // End of interrupt
    CPU0.ICCEOIR = (CPU0.ICCEOIR & ~(0x1FF)) | irq_num;
}
```

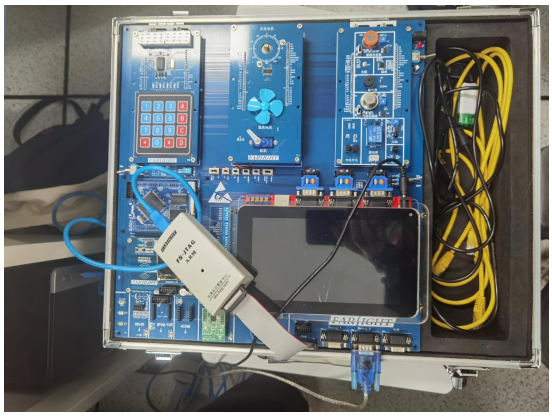
实验步骤:

1、 导入工程源码

参考 ARM 开发环境搭建部分。

光盘实验源码路径:华清远见-CORTEXA9 资料\实验代码\1. ARM 体系结构与接口技术部分\实验代码\05-Ket_Int

2、 连接好开发板及 FS-JTAG 仿真器，并且连接好配线。



3、 配置好串口终端

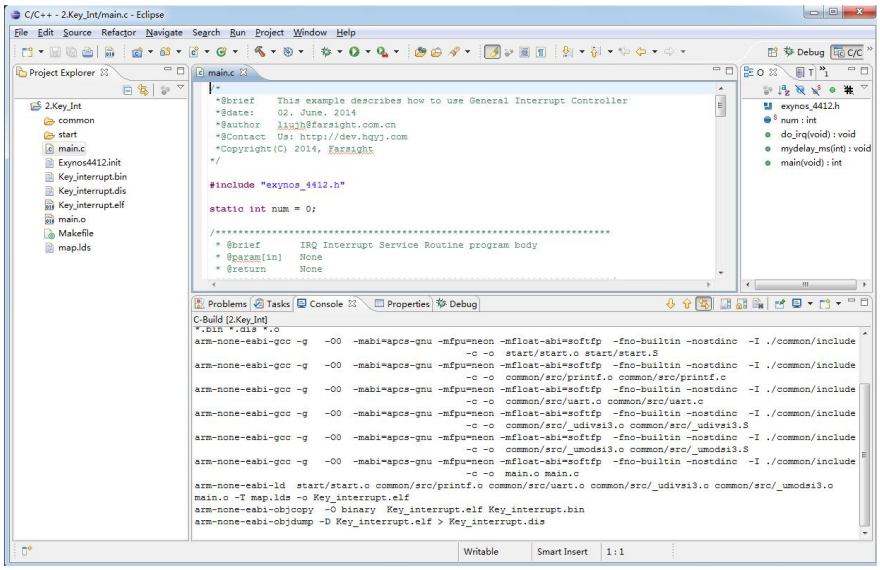
用于显示按键的值。方法参考 ARM 开发环境搭建部分。

4、仿真运行程序

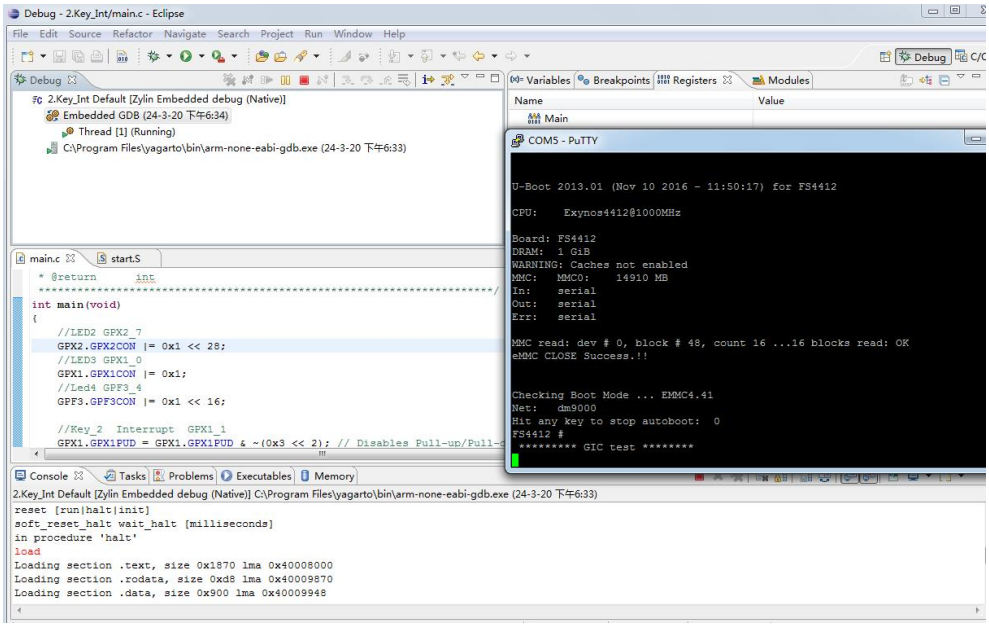
方法参考 ARM 开发环境搭建部分。

五、实验数据（现象）处理分析

1. 导入工程源码



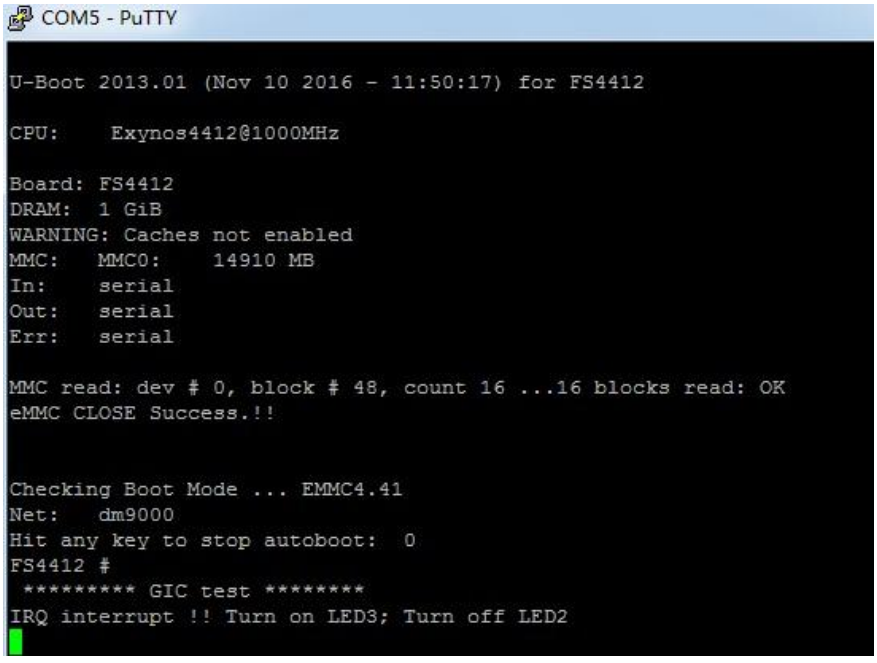
2. 配置好串口终端



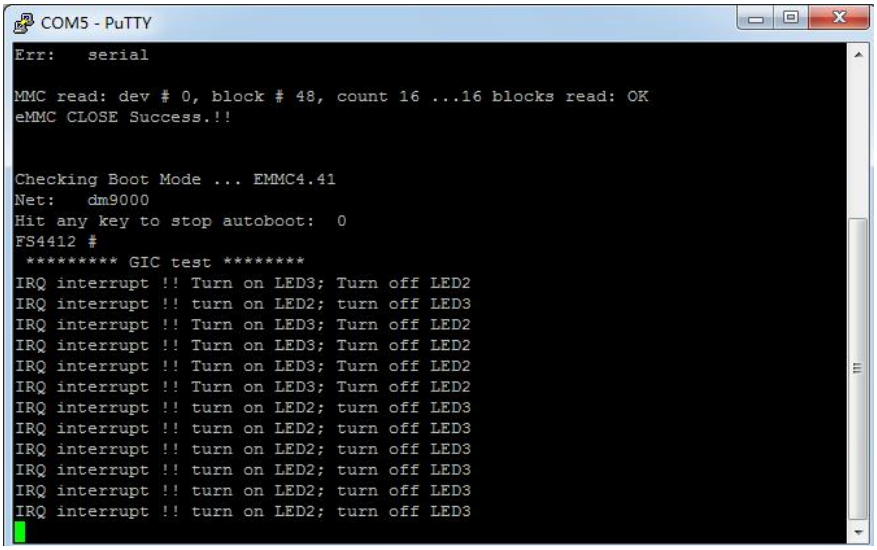
3. 仿真运行程序

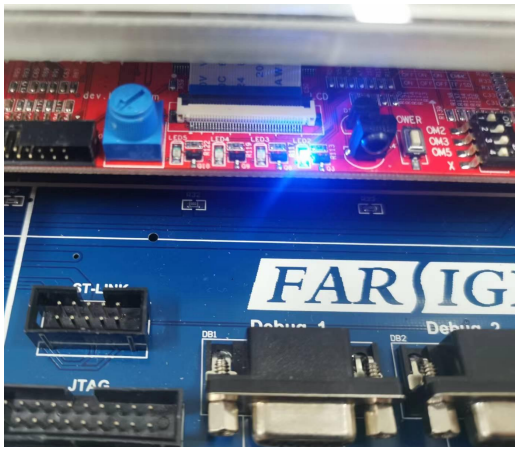
当按下 Key2 或 Key3 的时候观察 LED2 和 LED3 的亮灭情况，通过串口中断也可以看到对应的打印信息。

单独按下 Key3 键：



反复按 Key2，Key3 键，不断切换灯的亮灭状态。







六、实验结论

通过本次实验，深入了解了 Exynos4412 的中断处理机制。在配置了相应的 GPIO 引脚和中断参数后，成功实现了按键中断触发 LED 状态切换的功能。通过观察 LED 的状态和串口打印信息，验证了中断处理函数的正确性。此外，实验过程中对异常向量表的设置以及中断处理函数的编写也有了更深入的理解。

练习题

1．简述 ARM 有几种异常，以及每种异常对应的处理器工作模式。

异常	模式	用途
快速中断异常	FIQ	进行快速中断请求处理
外部中断请求	IRQ	进行外部中断请求处理
软中断异常	SVC	进行操作系统的高级处理
复位异常	SVC	进行操作系统的高级处理

预取指令中止异常	Abort	虚存和存储器保护
数据中止异常	Abort	虚存和存储器保护
未定义指令异常	Undefined	软件模拟硬件协处理器

2. 使用向量中断编写一个平台上其它按键中断的程序，并实现上升沿、下降沿、高电平、低电平等促发方式。

```
#include <avr/io.h>
#include <avr/interrupt.h>

// 定义按键所连接的引脚
#define BUTTON_PIN PD2

// 定义状态枚举
enum ButtonState {
    LOW_STATE,
    HIGH_STATE,
    RISING_EDGE,
    FALLING_EDGE
};

// 定义按键状态
volatile enum ButtonState buttonState = LOW_STATE;

int main(void) {
    // 设置 PD2 引脚为输入，并使能上拉电阻
    DDRD &= ~(1 << BUTTON_PIN);
    PORTD |= (1 << BUTTON_PIN);

    // 设置按键中断触发方式为下降沿触发
    EICRA |= (1 << ISC01);
    EIMSK |= (1 << INT0);

    // 全局中断使能
    sei();

    while (1) {
        // 在这里可以添加其他逻辑部分
    }
}

// 外部中断 0 的中断服务函数
ISR(INT0_vect) {
    // 读取按键引脚状态
```

```
    uint8_t buttonPinState = PIND & (1 << BUTTON_PIN);

    // 根据引脚状态确定按键状态
    if (buttonPinState) {
        if (buttonState == LOW_STATE)
            buttonState = RISING_EDGE;
        else
            buttonState = HIGH_STATE;
    }
    else {
        if (buttonState == HIGH_STATE)
            buttonState = FALLING_EDGE;
        else
            buttonState = LOW_STATE;
    }

    // 清除中断标志位
    EIFR |= (1 << INTF0);
}
```

七、教师批阅意见：

成绩评定：

教师签字：

年　　月　　日

八、备注：